



Escuela de Ingeniería en Computación
Ingeniería en Computación
IC6821 - Diseño de Software

PyStudio

Diseño de Arquitectura

Danny Jimenez Sevilla
danny.jimenez@estudiantec.cr
2020018418

Valeria Cascante Arce
v.cascante.1@estudiantec.cr
2024096746

Luna Antoniella Peraza Sandi
l.peraza.1@estudiantec.cr
2024071734

San Jose, Costa Rica
Abril 2026

Índice general

1. Introducción	2
1.1. Antecedentes y contexto del problema	2
1.2. Descripción del problema	3
1.3. Descripción de la solución propuesta	3
1.4. Justificación de la Solución	4
2. Diseño de Persistencia	5
2.1. Modelo Conceptual	6
2.2. Modelo Lógico	7
3. Diseño de Arquitectura	12
3.1. Diagrama de Componentes	14
3.2. Diagrama de Clases	22
3.2.1. Patrones de diseño identificados	22

Capítulo 1

Introducción

1.1. Antecedentes y contexto del problema

En el ámbito educativo costarricense y latinoamericano, la enseñanza de la programación en Python ha experimentado un crecimiento acelerado en los últimos años. Python se ha consolidado como el lenguaje introductorio preferido en universidades e instituciones técnicas debido a su sintaxis clara y su amplio ecosistema. A pesar de esto, existe la carencia de una herramienta que mezcle las capacidades de entornos de desarrollo presentes en el mercado como VS-Code o PyCharm con consideraciones de dinámicas pedagógicas propias de un aula de clases o un curso universitario y filtración de contenido generado por IA.

Actualmente, los docentes enfrentan una alta complejidad en asignaciones de naturaleza practica en áreas introductorios a la programación debido a dos razones principalmente:

1. La facilidad de acceso a la solución de problemas genéricos proveídos por IA, en el cual el factor principal de preocupación es la incapacidad futura del estudiante de generar contenido por su propia cuenta, los cuales son en mas de un 90 % de los casos coincide con el requerimiento de asignaciones.
2. La desconexión entre el entorno de gestión de entregas con el proceso interno de desarrollo de la entrega realizado por el estudiante en términos prácticos y colaborativos.

Estas condiciones generan fricción en el flujo de trabajo educativo y aprendizaje lo cual motiva la creación de **PyStudio**, una solución integrada que combina un entorno de desarrollo de escritorio con restricciones sobre el uso de IA, con una plataforma web de gestión académica para docentes unificando la experiencia de enseñanza y aprendizaje de Python.

1.2. Descripción del problema

El problema central que esta herramienta plantea resolver se puede entender por los siguientes puntos:

1. **Uso excesivo de IA:** Dado a la era en la que vivimos, el acceso a herramientas de inteligencia artificial ha crecido exponencialmente y con esto la constante vulnerabilidad al plagio o reducción de factor humano en asignaciones, mayor aun en el área de las ciencias de la computación.
2. **Fragmentación de herramientas:** Los estudiantes utilizan un IDEA para programar (en su mayoría con mas o menos facilidades genéricas y una plataforma separada para recibir y entregar actividades.
3. **Ausencia del soporte colaborativo integrado:** En muchos de los casos estudiantes que trabajan en grupo deben coordinarse fuera de su entorno de desarrollo, utilizando herramientas externas que no están diseñadas para la edición colaborativa de código.

1.3. Descripción de la solución propuesta

PyStudio es un sistema de software compuesto por dos componentes principales:

1. **Aplicación de escritorio para estudiantes:** Un IDE (Entorno de Desarrollo Integrado) para Python que permite a los estudiantes:
 - Escribir, editar y ejecutar código Python directamente en la aplicación.
 - Visualizar las actividades y asignaciones creadas por sus profesores.
 - Enviar sus soluciones directamente desde el IDE.
 - Unirse a cursos mediante códigos de acceso.
 - Formar grupos de trabajo con otros estudiantes del mismo curso.
 - Restringir la función de pegado.
2. **Aplicación web para docentes:** Una plataforma web que permite a los profesores:
 - Crear y gestionar cursos o salones de clase.
 - Diseñar actividades y asignaciones con instrucciones, fechas límite y recursos adjuntos.
 - Visualizar las entregas realizadas por los estudiantes.
 - Gestionar los miembros inscritos en cada curso.

Ambos componentes se conectan a un **servidor backend centralizado** que gestiona la autenticación, el almacenamiento de datos, la lógica de negocio y la comunicación entre las plataformas.

1.4. Justificación de la Solución

La construcción de PyStudio se justifica desde múltiples perspectivas:

1. **Desde la perspectiva pedagógica**, la integración del IDE con el flujo de actividades elimina la fricción en el proceso de aprendizaje. Los estudiantes pueden concentrarse en resolver problemas de programación sin preocuparse por la logística de entrega.
2. **Desde la perspectiva técnica**, separar la experiencia del estudiante (escritorio) de la del docente (web) permite optimizar cada interfaz para su público objetivo. La aplicación de escritorio ofrece un rendimiento superior para la edición, ejecución de código y implementación de restricciones de pegado de código, mientras que la plataforma web brinda accesibilidad y facilidad de uso desde cualquier navegador.
3. **Desde la perspectiva colaborativa**, la capacidad de formar grupos dentro del sistema fomenta el aprendizaje cooperativo y prepara a los estudiantes para las dinámicas de trabajo en equipo propias de la industria del software.

No existe actualmente una herramienta de código abierto que combine estas cuatro dimensiones en un producto unificado y orientado al contexto educativo latinoamericano.

Capítulo 2

Diseño de Persistencia

2.1. Modelo Conceptual

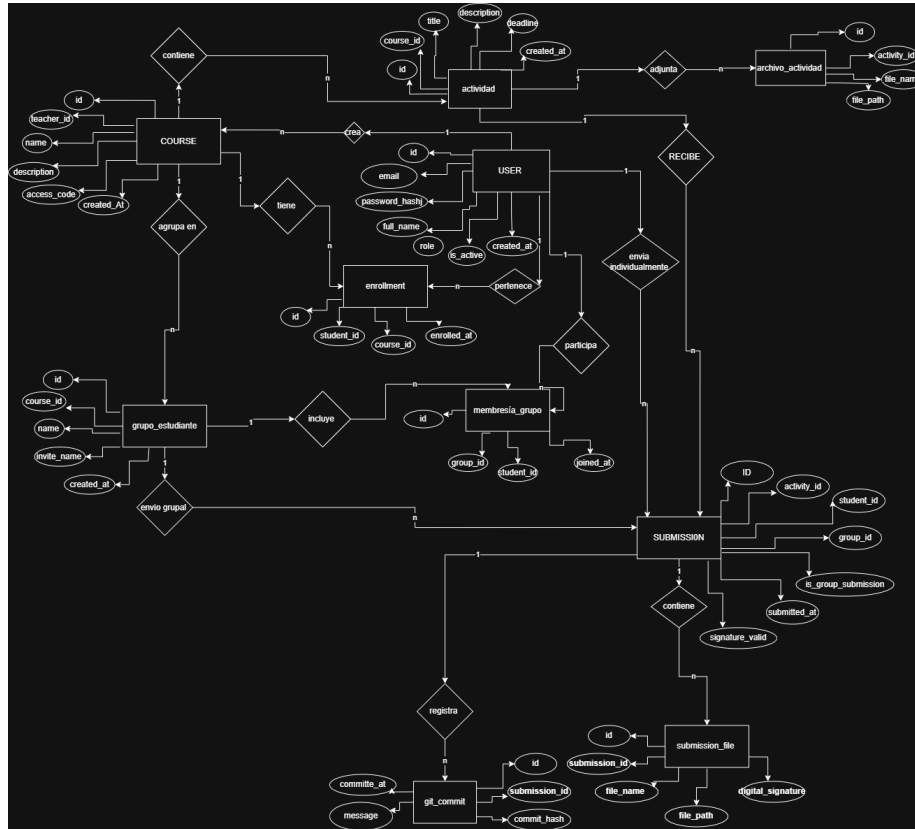


Figura 2.1: Modelo Conceptual

2.2. Modelo Lógico

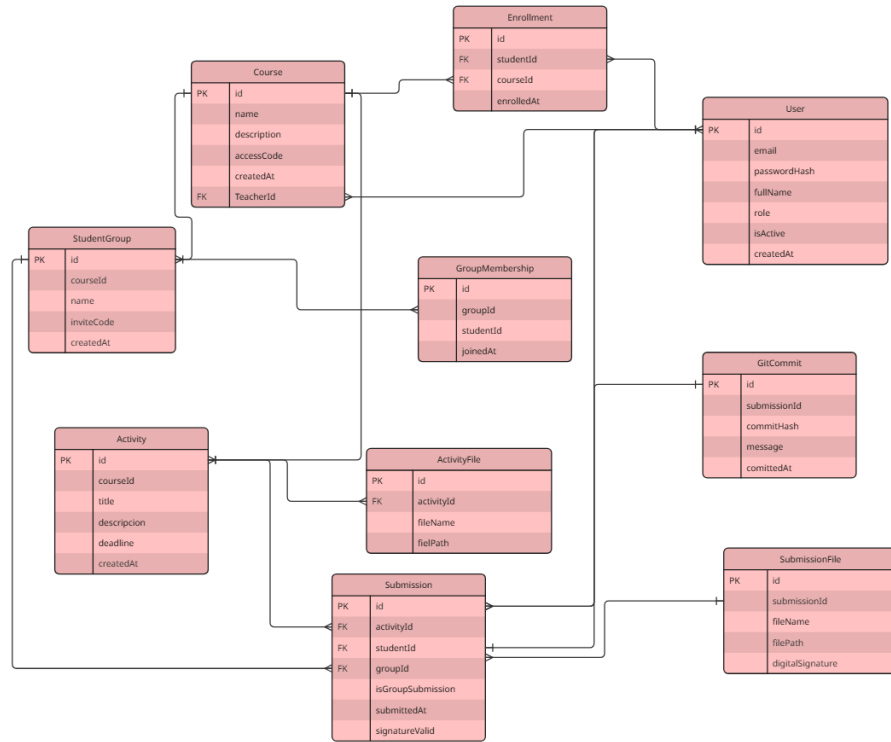


Figura 2.2: Modelo Lógico

Decisiones de normalización — 4FN

- **1FN** — Todos los atributos son atómicos. No existen grupos repetidos ni atributos multivalorados dentro de una misma fila.
- **2FN** — Todas las tablas tienen llave primaria simple (id autogenerated). No existen dependencias parciales.
- **3FN** — No existen dependencias transitivas. Cada atributo no-clave depende únicamente de la llave primaria.
- **4FN** — Se eliminaron las dependencias multivaloradas: **ACTIVITY_FILE** separa los adjuntos de actividades y **SUBMISSION_FILE** separa los archivos de entregas. La restricción de un estudiante por grupo por curso se aplica mediante lógica de aplicación, manteniendo la tabla **GROUP_MEMBERSHIP** en 4FN.

Diccionario de Datos

Para cada entidad se presenta una descripción y una tabla con los atributos del sistema.

USER

Representa a todos los usuarios registrados en el sistema (estudiantes y profesores). Es la entidad central de autenticación y control de acceso basado en roles.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
email	VARCHAR(255)	Correo electrónico válido RFC 5321	No	UQ · NN
password_hash	VARCHAR(255)	Hash bcrypt, nunca texto plano	No	NN
full_name	VARCHAR(150)	Cadena no vacía, máx. 150 chars	No	NN
role	ENUM	'student' o 'teacher'	No	NN
is_active	BOOLEAN	TRUE o FALSE, default TRUE	No	NN · DEFAULT TRUE
created_at	TIMESTAMP	Fecha y hora del registro	No	NN · DEFAULT NOW()

COURSE

Curso o salón de clase creado por un profesor. Contiene un código de acceso único que permite a los estudiantes inscribirse.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
teacher_id	INT	ID existente en USER con role=teacher	No	FK → USER · NN
name	VARCHAR(150)	Cadena no vacía, máx. 150 chars	No	NN
description	TEXT	Texto libre, puede estar vacío	Sí	—
access_code	VARCHAR(20)	Alfanumérico, generado por el sistema	No	UQ · NN
created_at	TIMESTAMP	Fecha y hora de creación	No	NN · DEFAULT NOW()

ENROLLMENT

Relación de inscripción entre un estudiante y un curso. Garantiza que un mismo estudiante no pueda inscribirse dos veces en el mismo curso mediante restricción UNIQUE sobre student_id y course_id.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
student_id	INT	ID existente en USER con role='student'	No	FK → USER · NN
course_id	INT	ID existente en COURSE	No	FK → COURSE · NN
enrolled_at	TIMESTAMP	Fecha y hora de la inscripción	No	NN · DEFAULT NOW()

ACTIVITY

Actividad académica publicada por un profesor dentro de un curso. Define el enunciado, fecha límite y puede tener archivos adjuntos asociados en ACTIVITY_FILE.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
course_id	INT	ID existente en COURSE	No	FK → COURSE · NN
title	VARCHAR(200)	Cadena no vacía, máx. 200 chars	No	NN
description	TEXT	Texto libre con instrucciones	Sí	—
deadline	TIMESTAMP	Fecha y hora futura al momento de creación	No	NN
created_at	TIMESTAMP	Fecha y hora de creación	No	NN · DEFAULT NOW()

ACTIVITY_FILE

Archivos adjuntos opcionales que el profesor agrega a una actividad (plantillas, PDFs, etc.). Tabla separada de ACTIVITY para eliminar dependencia multivalorada (4FN).

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
activity_id	INT	ID existente en ACTIVITY	No	FK → ACTIVITY · NN
file_name	VARCHAR(255)	Nombre del archivo con extensión	No	NN
file_path	VARCHAR(500)	Ruta en servidor de almacenamiento	No	NN

STUDENT_GROUP

Grupo de trabajo formado por estudiantes dentro de un curso. Genera un código de invitación único para que otros estudiantes puedan unirse.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
course_id	INT	ID existente en COURSE	No	FK → COURSE · NN
name	VARCHAR(100)	Cadena no vacía definida por el estudiante	No	NN
invite_code	VARCHAR(20)	Alfanumérico único, ej. GRP-A7X2	No	UQ · NN
created_at	TIMESTAMP	Fecha y hora de creación del grupo	No	NN · DEFAULT NOW()

GROUP_MEMBERSHIP

Relación de pertenencia entre un estudiante y un grupo. La restricción UNIQUE sobre student_id y group_id garantiza que un estudiante no se una dos veces al mismo grupo.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
group_id	INT	ID existente en STUDENT_GROUP	No	FK → STUDENT_GROUP · NN
student_id	INT	ID existente en USER con role='student'	No	FK → USER · NN
joined_at	TIMESTAMP	Fecha y hora en que el estudiante se unió	No	NN · DEFAULT NOW()

SUBMISSION

Registro de entrega de una actividad. Exactamente uno de student_id o group_id tiene valor según is_group_submission. Incluye estado de firma digital para verificación de integridad.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
activity_id	INT	ID existente en ACTIVITY	No	FK → ACTIVITY · NN
student_id	INT	ID en USER; NULL si es entrega grupal	Sí	FK → USER
group_id	INT	ID en STUDENT_GROUP; NULL si es individual	Sí	FK → STUDENT_GROUP
is_group_submission	BOOLEAN	TRUE=grupal, FALSE=individual	No	NN
submitted_at	TIMESTAMP	Marca de tiempo exacta del envío	No	NN

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
signature_valid	BOOLEAN	TRUE=íntegro, FALSE=alterado externamente	No	NN · DEFAULT TRUE

SUBMISSION_FILE

Archivos .py individuales de una entrega. Separados de SUBMISSION para eliminar dependencias multivaloradas (4FN). Cada archivo tiene su propia firma digital.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
submission_id	INT	ID existente en SUBMISSION	No	FK → SUBMISSION · NN
file_name	VARCHAR(255)	Nombre del archivo .py	No	NN
file_path	VARCHAR(500)	Ruta en almacenamiento del servidor	No	NN
digital_signature	TEXT	Hash firmado con clave privada del servidor	Sí	NULL solo si falla la firma (RF-19 A2)

GIT_COMMIT

Registro de un commit automático generado por el IDE cada vez que el estudiante guarda o ejecuta código. Permite al profesor visualizar la evolución temporal del desarrollo.

Atributo	Tipo SQL	Valores válidos	Null	Restricciones
id	INT	Entero positivo autogenerado	No	PK · AUTO_INCREMENT
submission_id	INT	ID existente en SUBMISSION	No	FK → SUBMISSION · NN
commit_hash	VARCHAR(64)	Hash SHA-1 o SHA-256 del commit Git	No	NN
message	VARCHAR(255)	Formato: [auto] YYYY-MM-DD HH:MM:SS	No	NN
committed_at	TIMESTAMP	Marca de tiempo del commit automático	No	NN

Capítulo 3

Diseño de Arquitectura

3.1. Diagrama de Componentes

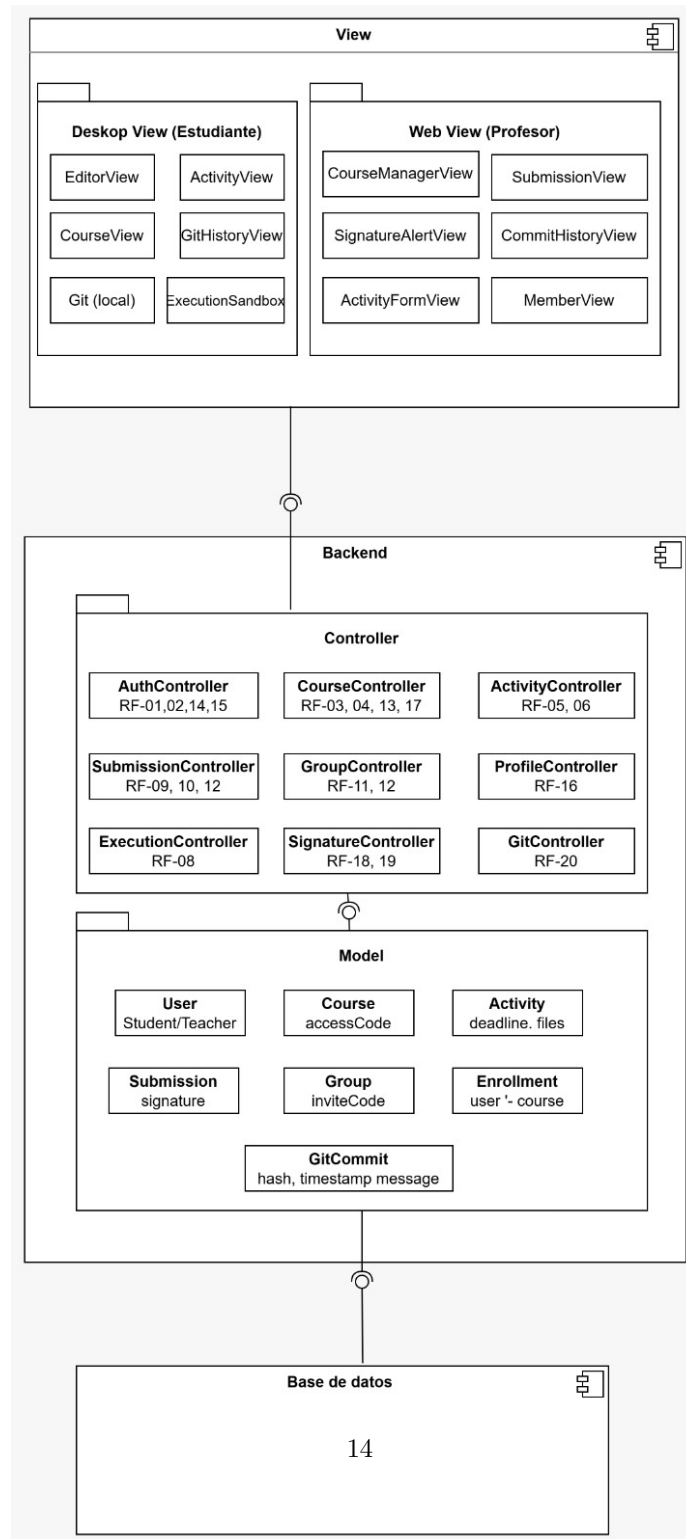


Figura 3.1: Diagrama de componentes

Desktop View (Estudiante)

El componente **Desktop View (Estudiante)** es la interfaz gráfica de la aplicación de escritorio con la que interactúa directamente el estudiante. Su rol principal es presentar la información recibida desde los controladores del backend y recibir las acciones del usuario para enviarlas al sistema. Este componente no contiene lógica de negocio; únicamente se encarga de la presentación y la navegación. Internamente agrupa las vistas **EditorView**, **ActivityView**, **CourseView** y **GitHistoryView**, así como los módulos locales **Git (local)** y **ExecutionSandbox** que operan directamente en la máquina del estudiante.

Método	Entrada	Salida	Ejemplo
renderEditor (file)	Objeto con nombre y contenido del archivo	Renderiza el editor con resaltado de sintaxis y numeración de líneas	Entrada: name: "ta-real.py". Abre el editor con el archivo cargado
renderActivities (list)	Lista de actividades del curso activo	Renderiza la lista de actividades con título y fecha límite	Entrada: id:1, title:"Tarea 1". Muestra la tarjeta de la actividad
renderCourses (list)	Lista de cursos inscritos por el estudiante	Renderiza el panel lateral con los cursos activos	Entrada: id:2, name:Íntro Python". Muestra el curso en el menú lateral
renderGitHistory (commits)	Lista de commits del proyecto activo	Renderiza el historial de versiones con hash, fecha y mensaje	Entrada: hash:"a1b2", message:"auto". Muestra la línea de tiempo de cambios
showExecution Output(result)	Objeto con stdout, stderr y returnCode	Renderiza la consola con la salida del programa ejecutado	Entrada: stdout:"Hola Mundo", returnCode:0. Imprime la salida en consola
showError (message)	Cadena de texto con el mensaje de error	Muestra un mensaje de error visible al usuario	Entrada: "Archivo no encontrado". Despliega un banner rojo

Web View (Profesor)

El componente **Web View (Profesor)** es la interfaz web accesible desde cualquier navegador que utilizan los profesores para administrar sus cursos y revisar las entregas de los estudiantes. Al igual que el Desktop View, este componente es exclusivamente de presentación y delega toda la lógica al backend. Agrupa las vistas **CourseManagerView**, **SubmissionView**, **SignatureAlertView**, **CommitHistoryView**, **ActivityFormView** y **MemberView**, cada una orientada a una tarea específica del flujo de trabajo del docente.

Método	Entrada	Salida	Ejemplo
renderCourse Manager(courses)	Lista de cursos del profesor	Renderiza el panel con opciones para crear, editar y eliminar cursos	Entrada: id:1, name:Íntro Python". Muestra el tablero de cursos
renderSubmissions (list)	Lista de entregas de una actividad	Renderiza entregas con nombre del estudiante, fecha y estado de firma	Entrada: student:"Maria", signatureValid:true. Muestra cada entrega con su indicador

Método	Entrada	Salida	Ejemplo
<code>renderSignatureAlert(submission)</code>	Entrega con <code>signatureValid</code> igual a <code>false</code>	Renderiza alerta indicando que el archivo fue modificado externamente	Entrada: <code>id:5</code> , <code>signatureValid:false</code> . Muestra un banner de advertencia rojo
<code>renderCommitHistory(commits)</code>	Lista de commits del proyecto de un estudiante	Renderiza historial con hash, fecha y mensaje para análisis	Entrada: <code>hash:ç3d4"</code> , <code>message:.^uto"</code> . Muestra la línea temporal de desarrollo
<code>renderActivityForm(activity)</code>	Objeto con datos actuales o vacío para creación nueva	Renderiza el formulario para crear o editar una actividad	Entrada: objeto vacío. Despliega el formulario para nueva actividad
<code>renderMembers(list)</code>	Lista de estudiantes inscritos en un curso	Renderiza tabla de miembros con nombre, correo y opción de remoción	Entrada: <code>id:3</code> , <code>name:"Pedro"</code> . Muestra la tabla de miembros del curso

AuthController

El componente **AuthController** es el controlador responsable de gestionar toda la identidad de los usuarios dentro de PyStudio. Centraliza el registro de nuevas cuentas, la autenticación mediante tokens JWT, el cierre de sesión y la recuperación de contraseña. Es el primer punto de contacto entre las vistas y el backend para cualquier operación que involucre credenciales o sesiones activas, garantizando que únicamente usuarios válidos y autenticados puedan acceder a los recursos protegidos del sistema.

Método	Entrada	Salida	Ejemplo
<code>register(data)</code>	Objeto con <code>name</code> , <code>email</code> , <code>password</code> y <code>role</code>	Objeto con <code>userId</code> y <code>token</code> , o error de validación	Entrada: <code>name:"Maria"</code> , <code>role:"student"</code> . Retorna <code>userId:12</code> y <code>token JWT</code>
<code>login(credentials)</code>	Objeto con <code>email</code> y <code>password</code>	Objeto con <code>token</code> y <code>role</code> , o error de autenticación	Entrada: <code>email:carlos@tec.cr"</code> . Retorna <code>token JWT</code> y <code>role:"teacher"</code>
<code>logout(token)</code>	Token JWT activo en el header de la solicitud	Objeto con <code>success</code> igual a <code>true</code> e invalidación del token	Entrada: <code>token JWT</code> activo. Invalida la sesión y retorna <code>success:true</code>
<code>recoverPassword(email)</code>	Correo electrónico registrado en el sistema	Mensaje de confirmación, o error si el correo no existe	Entrada: <code>"maria@tec.cr"</code> . Envía enlace y retorna <code>message:.Email enviado"</code>
<code>resetPassword(data)</code>	Objeto con <code>token</code> temporal y <code>newPassword</code>	Objeto con <code>success</code> igual a <code>true</code> , o error si el token expiró	Entrada: <code>token:.^bc123"</code> . Actualiza la contraseña del usuario

CourseController

El componente **CourseController** gestiona el ciclo de vida completo de los cursos dentro de PyStudio. Es responsable de la creación de nuevos cursos con generación automática de códigos de acceso únicos, la inscripción de estudiantes, la edición y eliminación de cursos existentes, y la gestión de los miembros

inscritos. Este controlador actúa como intermediario entre las vistas de gestión académica y los modelos **Course** y **Enrollment** en la base de datos.

Método	Entrada	Salida	Ejemplo
createCourse (data)	Objeto con name, description y teacherId	Objeto con courseId y accessCode generado automáticamente	Entrada: name:"Intro Python", teacherId:3. Retorna courseId:7 y accessCode:"PY101-2026"
enrollStudent (data)	Objeto con studentId y accessCode	Objeto con courseId y courseName, o error si el código es inválido	Entrada: studentId:12, accessCode:"PY101-2026". Retorna courseName:"Intro Python"
getCourses (userId)	Identificador entero del usuario	Lista de cursos según el rol del usuario autenticado	Entrada: userId:12. Retorna todos los cursos del estudiante con id 12
updateCourse (data)	Objeto con courseId, name y description	Objeto con success igual a true, o error de permisos	Entrada: courseId:7, name:"Python Avanzado". Actualiza el nombre del curso
deleteCourse (courseId)	Identificador entero del curso a eliminar	Objeto con success igual a true, o error si el curso no existe	Entrada: courseId:7. Elimina el curso y todas sus actividades asociadas
removeMember (data)	Objeto con courseId y studentId	Objeto con success igual a true, o error de permisos	Entrada: courseId:7, studentId:12. Elimina la inscripción del estudiante

ActivityController

El componente **ActivityController** es el encargado de administrar las actividades académicas dentro de cada curso. Permite a los profesores crear, editar y eliminar actividades con sus respectivas instrucciones, fechas límite y archivos adjuntos opcionales. También expone métodos para que los estudiantes puedan recuperar la lista de actividades disponibles en los cursos en los que están inscritos. Actúa como intermediario entre las vistas de actividades y el modelo **Activity** en la base de datos.

Método	Entrada	Salida	Ejemplo
createActivity (data)	Objeto con courseId, title, description y deadline	Objeto con activityId, o error de validación	Entrada: courseId:7, title:"Tarea 1". Retorna activityId:15
getActivities (courseId)	Identificador entero del curso	Lista de actividades publicadas en el curso indicado	Entrada: courseId:7. Retorna todas las actividades con título y fecha límite
getActivityDetail (activityId)	Identificador entero de la actividad	Objeto Activity con todos sus campos, o error si no existe	Entrada: activityId:15. Retorna title:"Tarea 1" y deadline:"2026-04-30"
updateActivity (data)	Objeto con activityId y campos a modificar	Objeto con success igual a true, o error de permisos	Entrada: activityId:15, deadline:"2026-05-05". Actualiza la fecha límite

Método	Entrada	Salida	Ejemplo
deleteActivity (activityId)	Identificador entero de la actividad	Objeto con success igual a true, o error si no existe	Entrada: activityId:15. Elimina la actividad y sus entregas asociadas

SubmissionController

El componente **SubmissionController** gestiona el proceso completo de entregas de actividades en PyStudio, tanto individuales como grupales. Recibe los archivos de código enviados por los estudiantes desde el IDE, registra la marca de tiempo del envío y los almacena en el servidor. También expone métodos para que los profesores puedan consultar y visualizar las entregas recibidas para cada actividad. Coordina con el **SignatureController** para garantizar que cada entrega quede firmada digitalmente al momento del envío.

Método	Entrada	Salida	Ejemplo
submitActivity (data)	Objeto con activityId, studentId y lista de archivos .py	Objeto con submissionId y timestamp, o error si no existe la actividad	Entrada: activityId:15, studentId:12. Retorna submissionId:33 y timestamp
submitGroupActivity (data)	Objeto con activityId, groupId y lista de archivos	Objeto con submissionId y timestamp, o error si el grupo ya entregó	Entrada: activityId:15, groupId:4. Registra la entrega grupal completa
getSubmissions (activityId)	Identificador entero de la actividad	Lista de entregas con nombre del autor, fecha y estado de firma	Entrada: activityId:15. Retorna todas las entregas recibidas para esa actividad
getSubmissionDetail (submissionId)	Identificador entero de la entrega	Objeto Submission con archivos, timestamp y signatureValid	Entrada: submissionId:33. Retorna los archivos y el estado de firma de la entrega

GroupController

El componente **GroupController** administra la formación y gestión de grupos de trabajo dentro de los cursos de PyStudio. Permite a los estudiantes crear grupos con un nombre personalizado, generar automáticamente un código de invitación único y unirse a grupos existentes mediante dicho código. Este controlador garantiza que un estudiante no pueda pertenecer a más de un grupo dentro del mismo curso y que los grupos solo puedan formarse en cursos que tengan habilitada esta funcionalidad.

Método	Entrada	Salida	Ejemplo
createGroup (data)	Objeto con courseId, studentId y name del grupo	Objeto con groupId e inviteCode generado automáticamente	Entrada: courseId:7, name:"Grupo Alpha". Retorna groupId:4 e inviteCode:"GRP-A7X2"

Método	Entrada	Salida	Ejemplo
joinGroup(data)	Objeto con studentId e inviteCode del grupo	Objeto con groupId y groupName, o error si el código es inválido	Entrada: studentId:14, inviteCode:"GRP-A7X2". Agrega al estudiante al Grupo Alpha
getGroupMembers(groupId)	Identificador entero del grupo	Lista de usuarios miembros del grupo indicado	Entrada: groupId:4. Retorna id:12 nombre:"Maria", id:14 nombre:"Pedro"
getStudentGroup(data)	Objeto con studentId y courseId	Objeto Group del estudiante en ese curso, o null si no pertenece a ninguno	Entrada: studentId:12, courseId:7. Retorna groupId:4 y name:"Grupo Alpha"

ProfileController

El componente **ProfileController** es responsable de gestionar la información personal de los usuarios registrados en PyStudio. Permite a cualquier usuario, independientemente de su rol, visualizar y actualizar sus datos de perfil como nombre, correo electrónico y contraseña. Valida que los nuevos datos cumplan con las reglas del sistema antes de persistirlos, verificando que el nuevo correo no esté en uso por otra cuenta y que la nueva contraseña cumpla los requisitos mínimos de seguridad.

Método	Entrada	Salida	Ejemplo
getProfile(userId)	Identificador entero del usuario	Objeto User con nombre, correo y rol del usuario	Entrada: userId:12. Retorna name:"Maria", email:"maria@tec.crz", role:"student"
updateProfile(data)	Objeto con userId, name y email a actualizar	Objeto con success igual a true, o error si el correo está duplicado	Entrada: userId:12, name:"Maria Lopez". Actualiza el nombre del perfil
changePassword(data)	Objeto con userId, currentPassword y newPassword	Objeto con success igual a true, o error si la contraseña actual es incorrecta	Entrada: userId:12, newPassword:"NuevaClave99!". Actualiza la contraseña

ExecutionController

El componente **ExecutionController** coordina la ejecución del código Python escrito por los estudiantes dentro del IDE de escritorio. Invoca el módulo **ExecutionSandbox** local para correr el código en un proceso aislado con límites de tiempo y memoria, previniendo que programas erróneos o maliciosos afecten el sistema del estudiante. Recibe el resultado de la ejecución y lo devuelve a la vista para mostrarlo en la consola integrada del IDE.

Método	Entrada	Salida	Ejemplo
<code>executeCode(data)</code>	Objeto con filePath y timeoutSeconds	Objeto con stdout, stderr y returnCode del proceso ejecutado	Entrada: filePath:/proyectos/tarea1.py", timeoutSeconds:10. Retorna stdout:"Hola Mundo" returnCode:0
<code>stopExecution(processId)</code>	Identificador entero del proceso en ejecución	Objeto con success igual a true al terminar el proceso forzosamente	Entrada: processId:4821. Detiene el proceso que superó el tiempo límite

SignatureController

El componente **SignatureController** garantiza la integridad de los archivos entregados por los estudiantes en PyStudio. Al momento de cada entrega, calcula un hash criptográfico del contenido de cada archivo .py y lo firma con la clave privada del servidor, almacenando la firma junto con la entrega. Cuando el profesor visualiza una entrega, este controlador recalcula el hash y lo compara con la firma almacenada para detectar si el archivo fue modificado fuera de la plataforma, marcándolo como inválido en caso de discrepancia.

Método	Entrada	Salida	Ejemplo
<code>signSubmission(data)</code>	Objeto con submissionId y lista de archivos de la entrega	Objeto con signature y signatureValid igual a true	Entrada: submissionId:33, files:"tarea1.py". Retorna signature:"3f9a...z" signatureValid:true
<code>verifySubmission(submissionId)</code>	Identificador entero de la entrega a verificar	Objeto con signatureValid true si es íntegro, o false si fue modificado externamente	Entrada: submissionId:33. Compara el hash y retorna signatureValid:true o false

GitController

El componente **GitController** administra el historial de versiones del código de los estudiantes dentro del IDE de escritorio. Se encarga de inicializar el repositorio Git local al abrir una actividad por primera vez y de realizar commits automáticos con marca de tiempo cada vez que el estudiante guarda o ejecuta su código, sin requerir intervención manual. Adicionalmente, expone métodos para que el backend pueda recuperar y sincronizar el historial de commits hacia la base de datos, permitiendo al profesor visualizar la evolución del código de cada entrega.

Método	Entrada	Salida	Ejemplo
<code>initRepository(projectPath)</code>	Cadena con la ruta local del proyecto	Objeto con success igual a true, o error si Git no está disponible	Entrada: /proyectos/tarea1". Ejecuta git init y retorna success:true
<code>autoCommit(data)</code>	Objeto con projectPath y timestamp del guardado	Objeto con commitHash del commit generado	Entrada: projectPath:/proyectos/tarea1". Retorna commitHash:."a1b2c3"

Método	Entrada	Salida	Ejemplo
<code>getCommitHistory(projectPath)</code>	Cadena con la ruta local del proyecto	Lista de GitCommit con hash, fecha y mensaje de cada commit	Entrada: /proyectos/tarea1". Retorna hash:.a1b2c3z message:.auto"
<code>syncCommits(data)</code>	Objeto con submissionId y lista de commits a persistir	Objeto con success igual a true al guardar el historial en la base de datos	Entrada: submissionId:33. Persiste el historial para consulta del profesor

3.2. Diagrama de Clases

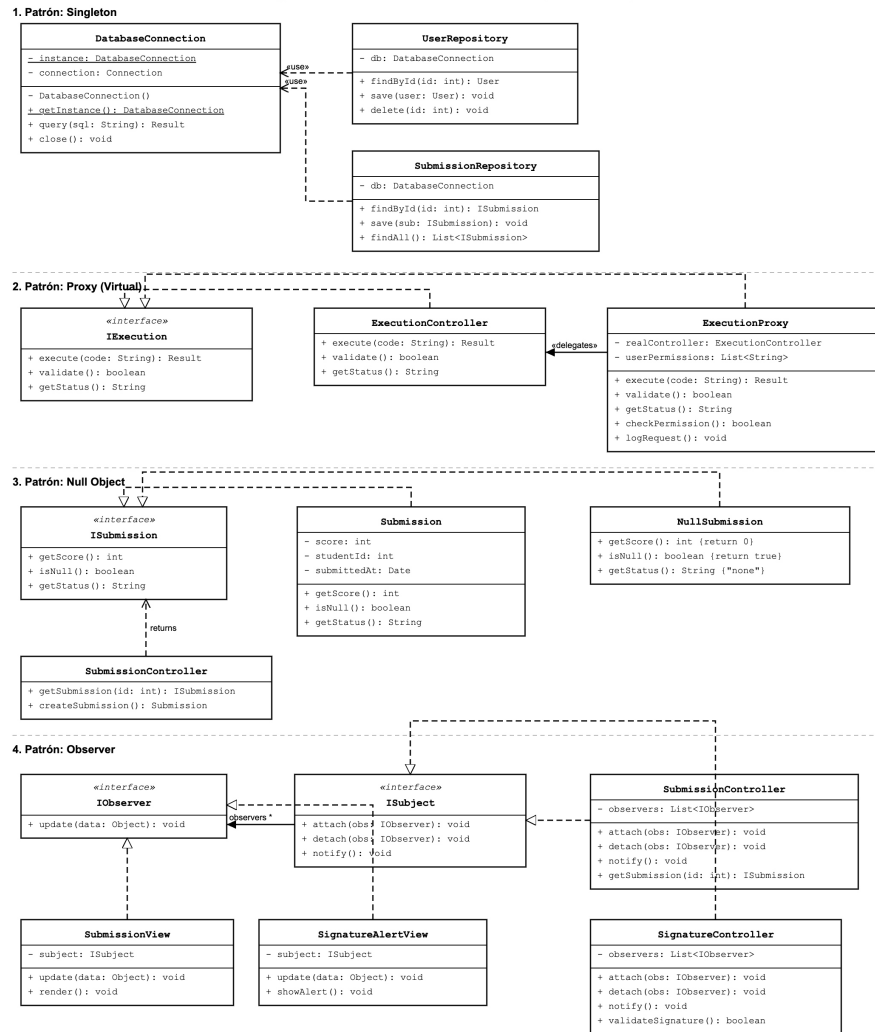


Figura 3.2: Diagrama de Clases

3.2.1. Patrones de diseño identificados

Singleton — DatabaseConnection

El patrón **Singleton** se implementó en el componente `DatabaseConnection`, el cual posee un constructor privado y un método estático `getInstance()`. Cual-

quier repositorio que necesite ejecutar queries obtiene siempre la misma instancia del pool de conexiones, evitando que múltiples conexiones queden abiertas simultáneamente hacia PostgreSQL. Este patrón se justifica porque la conexión a la base de datos es un recurso compartido y costoso; permitir múltiples instancias generaría inconsistencias y degradación del rendimiento en el sistema.

Proxy Virtual — ExecutionProxy

El patrón **Proxy Virtual** se implementó mediante el componente **ExecutionProxy**, el cual implementa la misma interfaz **IExecution** que el **ExecutionController** real. Cuando un controlador solicita ejecutar código, la petición pasa primero por el proxy, que valida permisos mediante **checkPermission()** y registra la solicitud mediante **logRequest()** antes de delegar la ejecución al **ExecutionController** real. El cliente no conoce con cuál de los dos componentes está interactuando. Este patrón se añadió para centralizar la validación de permisos y el registro de auditoría sin modificar la lógica del controlador de ejecución, respetando el principio de responsabilidad única.

Null Object — NullSubmission

El patrón **Null Object** se implementó en el flujo de búsqueda de entregas del **SubmissionController**. Cuando se busca la entrega de un estudiante y esta no existe, el sistema retorna una instancia de **NullSubmission** en lugar de **null**. La clase **NullSubmission** implementa la interfaz **ISubmission** con valores seguros por defecto (**score: 0, isNull(): true**), eliminando por completo la necesidad de verificaciones **if (submission == null)** en toda la capa de presentación. Este patrón se incorporó para reducir el acoplamiento entre capas y prevenir errores de referencia nula en las vistas.

Observer — SubmissionController y SignatureController

El patrón **Observer** se implementó en los componentes **SubmissionController** y **SignatureController**, ambos implementando la interfaz **ISubject** y manteniendo una lista de objetos **IObserver** registrados. Cuando llega una nueva entrega o se completa la firma digital de un archivo, estos controladores invocan **notify()** y las vistas **SubmissionView** y **SignatureAlertView** se actualizan automáticamente sin ningún acoplamiento directo entre la capa de control y la capa de presentación. Este patrón se eligió para desacoplar los controladores de las vistas que dependen de sus eventos, facilitando la extensibilidad del sistema ante nuevos observadores futuros.